

Prova scritta Programmazione Procedurale con Lab. - 1 Settembre 2026 —
CORREZIONE

Nome e Cognome: _____

Matricola: _____

1. 6 punti (soluzione nel riquadro) Cosa stampa il programma? Gli operatori unari hanno precedenza massima, a si trova all'indirizzo di memoria `0x7ffee2b4c9ac`, un `char` occupa 1 byte, uno `short` 2 byte, un `int` 4 byte. Supporre anche che, se l'operando sinistro di un operatore `&&` è falso (oppure quello di un operatore `||` è vero), l'operando destro NON viene valutato.

```
1 int a = 7, *b = &a;
2 int c = ((a-=4) || a--) && (a-=2, !a);
3 int d = !c && (a+=3, ((a-=1) && ++a));
4 int e = (c += 1, (d && c) &&
5         (d = a + 2, a--));
6 printf("%d %d %d %d\n", a, c, d, e);
7 printf("%p %p %lu\n", b, (int*)(
8         char*)b + 3, sizeof(*b));
```

3 1 6 1

0x7ffee2b4c9ac 0x7ffee2b4c9b8 4

Riga 2: $(a-=4) \rightarrow a=3$, vale 3 (vero) $\rightarrow ||$ corto-circuita e $a--$ NON viene valutato; l'operando sinistro di `&&` è vero $\rightarrow$ si valuta il destro: $a-=2 \rightarrow a=1$, $!a = !1 = 0 \rightarrow c = 0$.

Riga 3: $!c = !0 = 1$ (vero) $\rightarrow \&\&$ valuta il destro: $a+=3 \rightarrow a=4$; $(a-=1) \rightarrow a=3$, vero $\rightarrow ++a \rightarrow a=4$, vero $\rightarrow d = 1$.

Riga 4-5: $c+=1 \rightarrow c=1$; $(d \&\& c) = (1 \&\& 1) = 1$ (vero) $\rightarrow$ si valuta il destro: $d = a+2 = 6$, poi $a--$ ritorna 4 (vero), $a=3 \rightarrow e = 1$.

Stampa: 3 1 6 1.

Riga 7-8: $b = 0x7ffee2b4c9ac$; il cast $(char*)$ non sposta nulla, $(int*)b + 3$ avanza di 3 `int` = 12 byte $\rightarrow 0x7ffee2b4c9b8$ ($0xac + 0xc = 0xb8$); $sizeof(*b) = sizeof(int) = 4$.

2. 6 punti (su foglio protocollo) Scrivere una funzione che crea un array di n elementi (con n intero senza segno passato come parametro) e lo inizializza con la successione dei numeri triangolari (l' i -esimo numero triangolare è la somma dei numeri da 1 a i). Per esempio, se $n = 6$ l'array deve contenere 1-3-6-10-15-21. L'array creato deve essere poi ritornato dalla funzione come risultato. **(Soluzione: Esercizio 2, ultima pagina)**
3. 6 punti (su foglio protocollo) Data la seguente `struct Node` definire una funzione di nome `elimina_valore` che prende come parametro un valore di tipo `int`, x , ed elimina dalla lista (liberandone la memoria) il primo nodo che contiene x , lasciando la lista intatta se x non è presente. Per esempio, se la lista è 2-5-9 e x è 5, la nuova lista sarà 2-9. Considerare tutti i casi possibili (la lista può anche essere vuota, il nodo può essere in testa, in mezzo o in coda). Supporre di avere un puntatore ad inizio lista globale di nome `pFirst`. **(Soluzione: Esercizio 3, ultima pagina)**

```
1 struct Node {
2     int info;
3     struct Node* pNext;
4 };
```

4. 5 punti (su foglio protocollo) Dire quali compilazioni provocano errore a causa del linker (e perché): 1) `gcc -c main.c`, 2) `gcc -o main main.c`, 3) `gcc main.c out.c -o prog`, 4) `gcc -c out.c`, 5) `gcc -o out out.c`. In caso il punto 3) ritorni un errore, descrivere come può essere corretto. Infine, che tipo di *linkage* hanno k (in entrambi i file), w , e $mostra$, ed in quale file sono definite? Cosa stampa il programma? **(Soluzione: Esercizio 4, ultima pagina)**

```

1 /* main.c */
2 int k;
3 int k = 5;
4 void mostra(int k);
5
6 int main(void) {
7     extern int k;
8     while (k > 0) {
9         mostra(k);
10        k -= 2;
11    }
12    return 0;
13 }

```

```

1 /* out.c */
2 #include <stdio.h>
3 static int k = 10;
4 int w = 3;
5
6 void mostra(int a) {
7     a--;
8     printf("%d\n", k += w);
9 }

```

5. 7 punti (soluzione nel riquadro) Cerchiare le affermazioni vere dato $\underline{int\ a[4] = \{5+2*128, INT_MIN + 17, [2]=262148, 1048576/4+65\}; short\ int\ *p = (short*)\ a; char\ *q = (char*)\ a; *(q+1)=-1; *((short\ int*)\&q[10])=258;}$

sapendo che i tre tipi usati occupano 4, 2, e 1 byte, e $1048576 = 2^{20}$ (valori rappresentati in *little endian* e complemento a due). Scrivere la mappa di memoria e giustificare le affermazioni (vere o false).

A. $((\&a[2] - a) + p[4]) \% 3$ **[FALSA]** B. $((int)(a + 2) - (int)\&q[3]) + q[12]) \% 4$ **[VERA]** C. $((q[8] < 2) \mid q[10]) <= 18$ **[VERA]**

Mappa di memoria (un byte per riga; *little endian*, ogni byte rappresentato da sinistra verso destra, bit meno significativo a sinistra):

```

10100000 ← &q[0], &p[0] ; a[0] = 5+2*128 = 261 = 0x00000105 (byte 0 = 5)
11111111 ← *(q+1) = -1 sovrascrive il byte 1 → p[0] = 0xFF05 = -251 (complemento a due)
00000000 ← &p[1] (p[1]=0)
00000000

10001000 ← &p[2] (p[2]=17) ; a[1] = INT_MIN + 17 = 0x80000011 (byte 4 = 17)
00000000
00000000 ← &p[3] (p[3]=-32768)
00000001

00100000 ← &p[4] (p[4]=4) ; a[2] = 262148 = 1048576/4+4 = 218+4 = 0x00040004 (byte 8 = 4)
00000000
01000000 ← *((short*)&q[10]) = 258 = 0x0102 sovrascrive i byte 10-11 (q[10]=2, q[11]=1) → p[5] = 258
10000000

10000010 ← &p[6] (p[6]=65) ; a[3] = 1048576/4+65 = 262144+65 = 262209 = 0x00040041 (byte 12 = 65)
00000000
00100000 ← &p[7] (p[7]=4)
00000000

```

A) $((\&a[2] - a) + p[4]) \% 3 = (2 + 4) \% 3 = 6 \% 3 \rightarrow$ RISULTATO: 0 (**FALSA**). $\&a[2]-a = 2$ (differenza in numero di elementi int); $p[4] = 4$.
 B) $((int)(a+2) - (int)\&q[3]) + q[12]) \% 4 = (5 + 65) \% 4 = 70 \% 4 \rightarrow$ RISULTATO: 2 (**VERA**). $(int)(a+2)$ sta al byte 8, $(int)\&q[3]$ al byte 3 $\rightarrow$ differenza 5 byte; $q[12] = 65$.
 C) $((q[8] < 2) \mid q[10]) <= 18 : q[8] = 4 = 00000100b, 4 < 2 = 16 = 00010000b ; q[10] = 2 = 00000010b ; 16 \mid 2 = 18 ; 18 <= 18 \rightarrow$ RISULTATO: **VERA**.

Esercizio 2

```
1 #include <stdlib.h>
2
3 int *crea_triangu(unsigned int n) {
4     int *v = malloc(n * sizeof(int));    /* crea l'array di n elementi */
5     if (v == NULL)
6         return NULL;
7     for (unsigned int i = 0; i < n; i++) {
8         if (i == 0)
9             v[i] = 1;                    /* primo numero triangolare: 1 */
10        else
11            v[i] = v[i-1] + (i + 1);      /* precedente + (i+1) */
12    }
13    return v;                            /* l'array viene ritornato */
14 }
```

Esercizio 3

```
1 void elimina_valore(int x) {
2     if (pFirst == NULL)                /* lista vuota: niente da fare */
3         return;
4     if (pFirst->info == x) {            /* nodo da eliminare in testa */
5         struct Node *tmp = pFirst;
6         pFirst = pFirst->pNext;
7         free(tmp);
8         return;
9     }
10    struct Node *prev = pFirst;
11    while (prev->pNext != NULL && prev->pNext->info != x)
12        prev = prev->pNext;              /* prev: nodo che precede x */
13    if (prev->pNext != NULL) {            /* trovato: in mezzo o in coda */
14        struct Node *tmp = prev->pNext;
15        prev->pNext = tmp->pNext;
16        free(tmp);
17    }                                    /* se x non c'è, lista intatta */
18 }
```

Esercizio 4

- 1) `gcc -c main.c` → nessun errore di linker: `-c` compila soltanto (produce `main.o`), il linker non viene invocato.
- 2) `gcc -o main main.c` → ERRORE di linker: *mostra* non è definita (*undefined reference to mostra*).
- 3) `gcc main.c out.c -o prog` → nessun errore: la *k* di `out.c` è *static* (linkage interno), quindi non collide con la *k* di `main.c`; tutti i simboli si risolvono. (Non serve correzione.)
- 4) `gcc -c out.c` → nessun errore di linker: `-c` compila soltanto.
- 5) `gcc -o out out.c` → ERRORE di linker: *main* non è definito.

Linkage: *k* in `main.c` ha linkage ESTERNO (*int k*; è definizione tentativa, *int k = 5* la definizione), definita in `main.c`. *k* in `out.c` ha linkage INTERNO (*static*), definita in `out.c`: sono due oggetti DISTINTI. *w* ha linkage ESTERNO, definita in `out.c`. *mostra* ha linkage ESTERNO (dichiarata in `main.c`, definita in `out.c`). Il parametro *a* non ha linkage.

Output: in `main` la *k* globale di `main.c` parte da 5 e viene decrementata di 2 ad ogni iterazione (5, 3, 1, poi -1 e il ciclo termina): *mostra* viene chiamata 3 volte. Ad ogni chiamata *mostra* esegue $k += w$ sulla propria *k* static di `out.c`: 10+3=13, poi 16, 19. Il programma TERMINA (le due *k* sono variabili diverse).

13
16
19